



Для выполнения практики я беру **Artificial Analysis** как единый источник сравнения. Это удобно именно для нашей задачи, потому что на одной площадке есть не только общий рейтинг моделей, но и отдельные показатели для coding/agent-задач, цена токенов, скорость, latency и размер контекстного окна. В актуальной методологии общий Intelligence Index включает существенную долю агентных и coding-задач, а отдельный Coding Agent Index проверяет модели на DeepSWE, Terminal-Bench и SWE-Atlas-QnA. (artificialanalysis.ai)

Сценарий 1. Быстрый автокомплит

Начнём с автокомплита. Здесь я сразу меняю логику выбора модели. Мне не нужна самая интеллектуальная модель из рейтинга. Мне нужна модель, которая успеет предложить продолжение раньше, чем разработчик самостоятельно напишет следующую строку. Поэтому главным показателем становится latency, затем стоимость, и только после этого я смотрю на общий уровень модели.

Для этого сценария я выберу **GPT-5.6 Luna в режиме non-reasoning**. По данным Artificial Analysis, время до первого токена у этой модели составляет около **0,83 секунды**, а скорость генерации — около **133 токенов в секунду**. Стоимость составляет **\$0,20 за миллион входных токенов и \$1,20 за миллион выходных токенов**. При этом модель имеет контекстное окно в **1 миллион токенов**. (artificialanalysis.ai)

Почему здесь мне интересен именно non-reasoning режим? Потому что при обычном автодополнении нам не требуется, чтобы модель долго анализировала задачу. Если разработчик написал начало простого цикла, условного выражения или небольшой функции, система должна максимально быстро определить вероятное продолжение.

Для сравнения посмотрим на DeepSeek V4.1 Flash. На первый взгляд он выглядит даже привлекательнее: около **207 токенов в секунду**, контекст также **1 миллион токенов**, цена составляет **\$0,30 за миллион входных и \$1,20 за миллион выходных токенов**. Но это reasoning-вариант. Artificial Analysis показывает около **1,15 секунды до первого токена**, однако до первого именно ответного токена после внутреннего рассуждения проходит около **10,79 секунды**. Для интерактивного автокомплита это уже очень большая задержка. (artificialanalysis.ai)

Вот здесь хорошо видно, почему нельзя смотреть только на показатель tokens per second. Модель может очень быстро генерировать текст после начала ответа, но слишком долго начинать сам ответ. Для автокомплита разработчику гораздо важнее получить первые несколько полезных токенов практически мгновенно.

Контекстное окно в миллион токенов здесь фактически является избыточным преимуществом. По условиям нашего кейса обычно передаётся от одной до восьми тысяч токенов. Получается, что огромный контекст есть, но именно для этого сценария он почти не влияет на решение.

Поэтому мой выбор для первого кейса — GPT-5.6 Luna Non-reasoning. Я выбираю её не потому, что она сильнее всех моделей в абсолютном рейтинге, а потому, что сочетание низкой задержки, сравнительно высокой скорости и низкой стоимости лучше соответствует массовому автокомплиту.

Сценарий 2. Глубокий рефакторинг

Во втором кейсе приоритеты меняются практически полностью. Нам необходимо работать примерно с 80 тысячами токенов кода, учитывать зависимости между десятками файлов, сохранять поведение API и не ломать существующие тесты.

Здесь ответ через одну секунду мне уже не нужен. Если модель потратит существенно больше времени, но лучше поймёт взаимосвязи между компонентами, это приемлемый компромисс.

Для такого сценария я выберу **GPT-6 Astra в максимальном reasoning-режиме**. На текущем Artificial Analysis модель имеет **53 балла Intelligence Index** и контекстное окно в **1 миллион токенов**. Стоимость значительно выше лёгких моделей: **\$10 за миллион входных и \$50 за миллион выходных токенов**. Скорость генерации находится примерно на уровне **52 токенов в секунду**. (artificialanalysis.ai)

На первый взгляд цена высокая. Поэтому мне необходимо доказать, что мы действительно получаем что-то взамен.

Для нашего сценария гораздо интереснее отдельный **Artificial Analysis Coding Agent Index**, потому что он проверяет способность решать реальные задачи с программным кодом. В актуальном сравнении GPT-6 Astra в агентной coding-среде получает **62 балла**. Claude Fable 5.1 также получает 62 балла. То есть по этому индексу они находятся на одном уровне. При этом стоимость выполнения benchmark-задачи для Astra составляет около **\$7,47**, а для Fable 5.1 — около **\$12,39**. (artificialanalysis.ai)

Именно поэтому я не выбираю модель только по принципу «возьмём лидера общего рейтинга». Я смотрю на профиль нашей задачи. Нам требуется сложная работа с кодом, и в специализированном coding-бенчмарке Astra находится на верхнем уровне, но обходится заметно дешевле другого варианта с тем же итоговым показателем. Artificial Analysis отдельно отмечает, что Astra сравнялась с Fable 5.1 как в основном Intelligence Index, так и в Coding Agent Index, при меньшей стоимости. (artificialanalysis.ai)

Контекст в миллион токенов для нашего кейса более чем достаточен: исходный материал занимает около 80 тысяч токенов. Но я бы всё равно не передавал модели вообще всё содержимое проекта без разбора. Чем больше постороннего контекста мы добавляем, тем дороже становится запрос и тем больше нерелевантной информации приходится анализировать.

Мы также принимаем высокий latency как осознанный компромисс. Максимальный reasoning-режим Astra может начинать отвечать очень долго. Для автокомплита это было бы неприемлемо, но глубокий рефакторинг выполняется несколько раз в месяц, а не сотни раз в день. Здесь я готов заплатить временем и стоимостью за более сильный анализ.

Поэтому для второго кейса мой выбор — GPT-6 Astra с высоким уровнем reasoning. Главным аргументом становится результат специализированного coding-бенчмарка, затем качество общего анализа и размер контекста. Скорость в этом сценарии опускается значительно ниже по приоритету.

Сценарий 3. Анализ большой кодовой базы

Теперь у нас репозиторий примерно из семи тысяч файлов. Здесь появляется принципиально другая проблема: кодовая база может не помещаться даже в очень большое контекстное окно.

Поэтому я сразу отказываюсь от идеи искать модель, в которую мы просто загрузим весь репозиторий. Даже миллион токенов не означает, что необходимо каждый раз заполнять миллион токенов. Намного рациональнее сначала определить релевантные файлы и участки кода, а затем передавать их модели.

Для этого сценария я выберу **Muse Spark 1.3 в max-режиме**. По данным Artificial Analysis, модель имеет **48 баллов Intelligence Index**, контекстное окно в **1 миллион токенов** и генерирует около **224 токенов в секунду**. Стоимость составляет **\$1,25 за миллион входных** и **\$4,25 за миллион выходных токенов**. (artificialanalysis.ai)

Обратите внимание на разницу с предыдущим кейсом. GPT-6 Astra сильнее по общему рейтингу, но её входные токены стоят \$10 за миллион. У Muse Spark 1.3 вход стоит \$1,25. Для системы, которая регулярно извлекает и передаёт большие фрагменты репозитория, эта разница становится очень существенной. (artificialanalysis.ai)

При этом мы не переходим на совсем слабую дешёвую модель. Muse Spark 1.3 остаётся достаточно сильной моделью и показывает серьёзные результаты в задачах работы с кодом. В текущем Coding Agent Index связка с этой моделью получает **54 балла**, причём на DeepSWE результат достигает **72%**. (artificialanalysis.ai)

Есть недостаток: latency у max-версии высокий, около **26,7 секунды до первого токена**. Но в нашем сценарии пользователь не ждёт автокомплит во время набора строки. Мы выполняем комплексный анализ репозитория, поэтому задержка начала ответа для нас менее критична. (artificialanalysis.ai)

Самое важное здесь заключается в том, что окно в миллион токенов не решает задачу само по себе. Если кодовая база больше этого объёма, нам всё равно необходим механизм отбора контекста. Например, сначала можно определить файлы, связанные с платёжным API, затем найти импорты, вызовы соответствующих методов, конфигурацию и тесты и только после этого передавать релевантные материалы модели.

Получается, что модель с самым большим контекстным окном автоматически не становится лучшей моделью для анализа репозитория. Если одна модель немного сильнее, но обработка каждого миллиона входных токенов обходится в несколько раз дороже, регулярный анализ большой кодовой базы быстро становится дорогим.

Поэтому для третьего сценария мой выбор — Muse Spark 1.3. Здесь я сознательно выбираю баланс: большое контекстное окно, высокий уровень общего анализа, сильные показатели на coding-задачах, высокая скорость генерации и заметно более низкая стоимость обработки больших входных контекстов по сравнению с самым дорогим frontier-вариантом.

В результате у нас получаются три разные модели не потому, что одна «хорошая», а остальные «плохие». Для быстрого автокомплита я выбираю GPT-5.6 Luna Non-reasoning из-за минимальной задержки и низкой цены. Для глубокого рефакторинга выбираю GPT-6 Astra, потому что здесь макси-

мальный приоритет получает качество сложной работы с кодом. Для анализа очень большой кодовой базы выбираю Muse Spark 1.3, потому что здесь необходимо одновременно учитывать качество, миллионное контекстное окно и стоимость многократной передачи больших объёмов информации.

Именно так я хочу, чтобы мы подходили к выбору модели в реальном проекте: сначала определяем профиль нагрузки и критерии задачи, а уже после этого открываем benchmark и смотрим, какая модель лучше соответствует этим критериям. Самая высокая позиция в общем рейтинге сама по себе ещё не является достаточным основанием для выбора.

Все показатели здесь актуализированы по Artificial Analysis на сентябрь 2026 года; у рейтингов и цен есть свойство меняться, поэтому в реальной практике такую проверку имеет смысл выполнять непосредственно перед выбором модели. (artificialanalysis.ai)
